Programmablaufpläne mit Ti*k*Z erstellen

Fortbildungsmaterial für MINT-Lehrkräfte

13. Mai 2025

Programmablaufpläne (auch Flussdiagramme genannt) sind ein grundlegendes Werkzeug in der Informatik, um Algorithmen und Prozesse zu visualisieren. Mit dem Paket Ti*k*Z in LaTeX lassen sich solche Diagramme präzise, einheitlich und flexibel gestalten. Diese Handreichung zeigt, wie Sie mithilfe von Ti*k*Z ansprechende Programmablaufpläne erstellen können, indem Sie vordefinierte Stile nutzen und die Beziehungen zwischen den einzelnen Elementen definieren.

1 Grundlagen: Das Ti*k*Z-Paket

Ti*k*Z (Ti*k*Z ist kein Zeichenprogramm) ist ein leistungsstarkes Paket zur Erstellung von Vektorgrafiken in LaTeX. Um Ti*k*Z zu verwenden, müssen Sie zunächst das Paket einbinden und die benötigten Bibliotheken aktivieren. Für die Erstellung von Programmablaufplänen sind besonders die folgenden Bibliotheken relevant:

```
\usepackage{tikz}
\usetikzlibrary{shapes.geometric, arrows, positioning}
```

Das Hauptpaket `tikz` stellt die Grundfunktionen bereit, während die zusätzlichen Bibliotheken spezifische Funktionalitäten ergänzen. Mit `shapes.geometric` erhalten Sie Zugriff auf geometrische Formen wie Rauten für Entscheidungen oder Trapeze für Ein- und Ausgabeoperationen. Die `arrows`-Bibliothek bietet verschiedene Pfeilspitzen und -stile für die Verbindungslinien. Besonders wichtig für Programmablaufpläne ist die `positioning`-Bibliothek, die eine intuitive relative Positionierung der Knoten ermöglicht. Statt mit absoluten Koordinaten zu arbeiten, können Sie so Elemente mit Angaben wie „unterhalb von“, „rechts von“ usw. platzieren, was die Erstellung und spätere Anpassung des Diagramms erheblich vereinfacht. Diese Pakete sind bereits in der `header.tex` enthalten, sollten aber für das Verständnis erwähnt werden, da sie die Grundlage für alle weiteren Schritte bilden.

2 Eigene Stile für Programmablaufpläne definieren

Eine der Stärken von Ti*k*Z ist die Möglichkeit, eigene Stile zu definieren, die dann wiederverwendet werden können. Mit dem Befehl `\tikzstyle{name}` wird ein neuer Stil mit dem angegebenen Namen definiert, der dann für verschiedene Elemente in Ihren Diagrammen verwendet werden kann.
Schauen wir uns zunächst ein einfaches Beispiel an, wie ein Stil definiert wird:

```
% Definition eines einfachen Trapez-Stils
\tikzstyle{meinTrapez}=[
  trapezium,                  % Formvorgabe
  fill=blue!10,               % Fuellfarbe (hellblau)
  draw=blue,                  % Rahmenfarbe (blau)
  trapezium left angle=70,    % Winkel des linken Schraegstichs
  trapezium right angle=110   % Winkel des rechten Schraegstichs
]
```

Die eckigen Klammern enthalten alle Eigenschaften des Stils. Dies beginnt typischerweise mit der grundlegenden Form wie `rectangle`, `diamond` oder `trapezium`. Danach folgen Gestaltungsmerkmale wie die Füllfarbe (`fill=`) und die Rahmenfarbe (`draw=`). Je nach gewählter Form können zusätzliche Parameter erforderlich sein – beim Trapez beispielsweise die Winkel der schrägen Seiten.

Für Programmablaufpläne benötigen wir verschiedene Knotentypen, die den üblichen Konventionen entsprechen: Eingabeoperationen werden meist als grüne Parallelogramme dargestellt, Verarbeitungsschritte als orangefarbene Rechtecke, Entscheidungen als magentafarbene Rauten und Ausgabeoperationen als rote Parallelogramme. Diese Stile können wir wie folgt definieren:

```
%Tikzstyles fuer Programmablaufplaene
%Stil fuer Eingabe
\tikzstyle{input}=[
  trapezium,
  trapezium left angle=60,
  trapezium right angle=120,
  fill=green!20,
  draw=green
]
%Stil fuer Verarbeitung
\tikzstyle{operation}=[
  rectangle,
  fill=orange!20,
  draw=orange,
]
%Stil fuer Bearbeitung
\tikzstyle{condition}=[
  diamond,
  fill=magenta!20,
  draw=magenta,
]
Stil fuer Ausgabe
\tikzstyle{output}=[
  trapezium,
  trapezium left angle=60,
  trapezium right angle=120,
  fill=red!20,
  draw=red,
]
```

Mit diesen vordefinierten Stilen kann nun jeder Knoten im Programmablaufplan dem entsprechenden Typ zugewiesen werden. So entstehen visuell ansprechende und sofort verständliche Diagramme, die den standardisierten Konventionen folgen. Die Verwendung solcher Stile hat den großen Vorteil, dass Sie das Erscheinungsbild aller Elemente eines Typs durch Änderung der Stildefinition zentral anpassen können, ohne jeden einzelnen Knoten im Diagramm modifizieren zu müssen.

3 Grundlegendes zur Node-Funktion in Ti*k*Z

Die `\node`-Funktion ist der zentrale Baustein für Programmablaufpläne in Ti*k*Z. Sie definiert einen Knoten mit Position, Inhalt und Stil:

```
\node[stilname] (knotenname) at (x,y) {Knoteninhalt};
```

Die wichtigsten Elemente sind:

- `stilname`: Der zuvor definierte Stil, z.B. `operation`
- `(knotenname)`: Ein eindeutiger Name für den Knoten, der später zur Referenzierung dient
- `at (x,y)`: Die Position des Knotens im Koordinatensystem
- `{Knoteninhalt}`: Der Text, der im Knoten angezeigt wird

3.1 Knoten in Abhängigkeit voneinander positionieren

Statt absolute Koordinaten zu verwenden, ist es oft praktischer, Knoten relativ zu anderen zu positionieren:

```
% Erster Knoten mit absoluter Position
\node[operation] (start) at (0,0) {Start};

% Zweiter Knoten relativ zum ersten, 2cm unterhalb
\node[input] (eingabe) [below=2cm of start] {Eingabe};
```

Für diese relative Positionierung benötigen wir die Ti*k*Z-Bibliothek `positioning`. Die gängigsten Positionierungsangaben sind:

- `above`: oberhalb
- `below`: unterhalb
- `left`: links
- `right`: rechts
- Kombinationen wie `above right`: oberhalb rechts

Der Abstand wird mit `=2cm` angegeben und `of knotenname` gibt den Bezugsknoten an.

3.2 Verbindungspfeile zwischen Knoten

Um Knoten miteinander zu verbinden, verwendet man die `\draw`-Funktion:

```
\draw[->] (start) -- (eingabe);
```

Diese Anweisung zeichnet einen Pfeil vom Knoten `start` zum Knoten `eingabe`. Der Pfeilstil `->` gibt an, dass am Ende ein Pfeilkopf gezeichnet wird.

3.3 Anpassung der Knotenabstände

Der Parameter `node distance` legt den Standardabstand zwischen Knoten fest:

```
\begin{tikzpicture}[node distance=1.5cm] % Engerer Abstand
  % ...
\end{tikzpicture}
```

3.4 Beschriftung von Verbindungspfeilen

Pfeile können mit der `node`-Anweisung beschriftet werden:

```
\draw[->] (knoten1) -- node[above] {Beschriftung} (knoten2);
```

Die Positionsangabe (`above`, `below`, `left`, `right`) bestimmt, wo die Beschriftung platziert wird.

3.5 Komplexere Pfade

Für komplexere Verbindungen können wir Pfade definieren:

```
% Horizontale und dann vertikale Verbindung
\draw[->] (knoten1) -| (knoten2);

% Vertikale und dann horizontale Verbindung
\draw[->] (knoten1) |- (knoten2);

% Verbindung mit Versatz
\draw[->] (knoten1) -- ++(2,0) |- (knoten2);
```

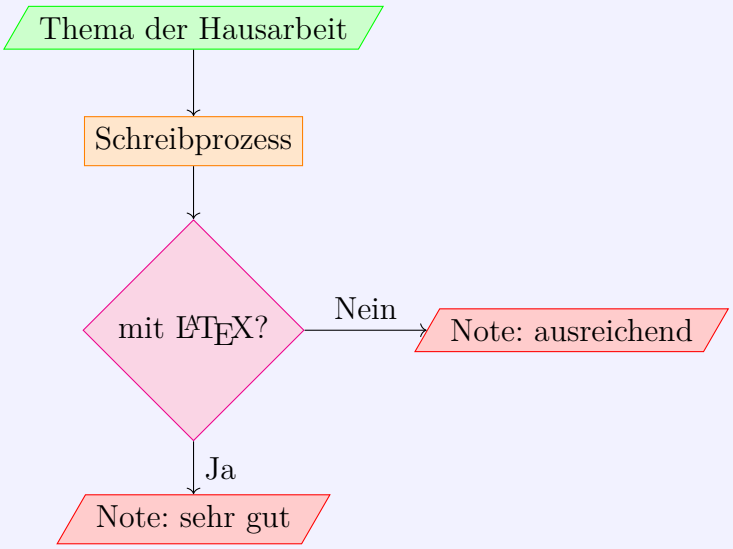
4 Ein einfaches Beispiel

Hier ein vollständiges Beispiel eines einfachen Programmablaufplans:

```
\begin{tikzpicture}[node distance=2.5cm]
  % Knoten definieren
  \node[input] (Eingabe) {Thema der Hausarbeit};
  \node[operation, below of=Eingabe, node distance=1.5cm] (Operation1) {Schreibprozess};
  \node[condition, below of=Operation1] (Bedingung1) {mit \hologo{LaTeX}?};
  \node[output, below of=Bedingung1] (Ausgabe1) {Note: sehr gut};
  \node[output, right of=Bedingung1, node distance=5cm] (Ausgabe2) {Note: ausreichend};

  % Pfeile zwischen den Knoten
  \draw[->] (Eingabe) -- (Operation1);
  \draw[->] (Operation1) -- (Bedingung1);
  \draw[->] (Bedingung1) -- node[right] {Ja} (Ausgabe1);
  \draw[->] (Bedingung1) -- node[above] {Nein} (Ausgabe2);
\end{tikzpicture}
```

Ausgabe



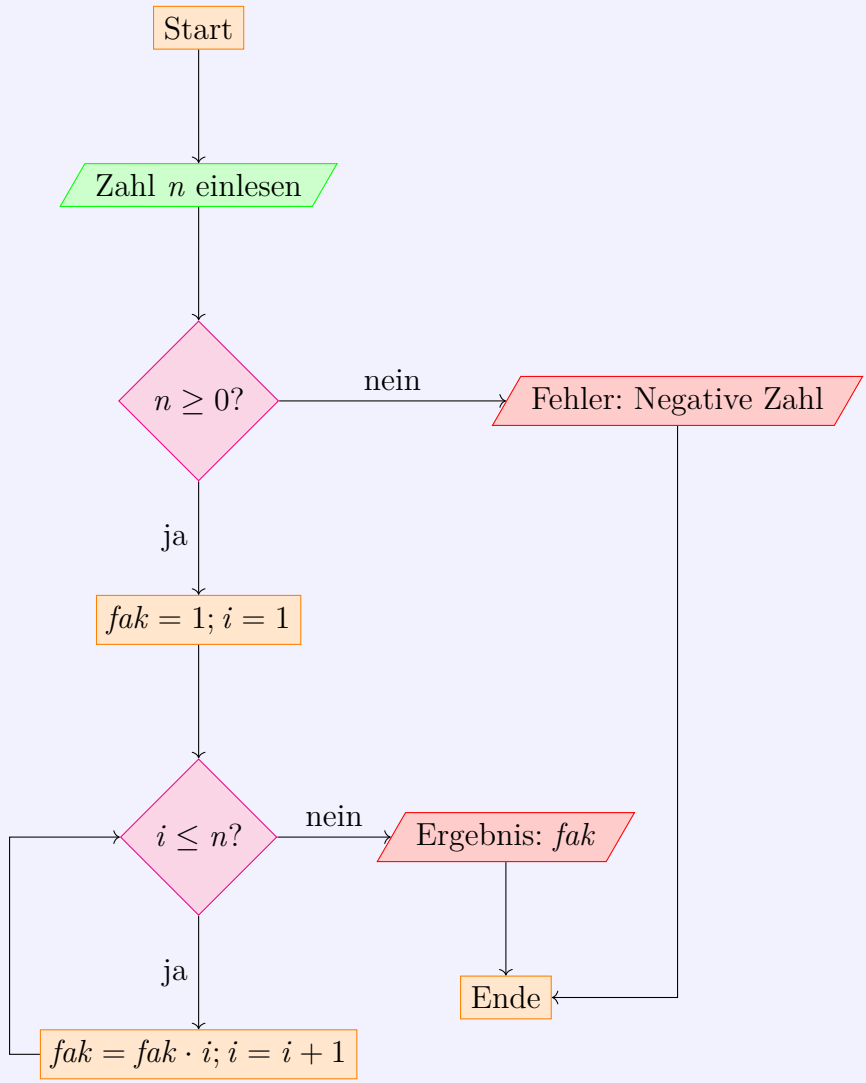
5 Ein komplexeres Beispiel

Hier ist ein komplexeres Beispiel eines Programmablaufplans für die Berechnung der Fakultät einer Zahl:

```
\begin{tikzpicture}[node distance=1.5cm]
  % Knoten definieren
  \node[operation] (start) {Start};
  \node[input] (eingabe) [below=of start] {Zahl $n$ einlesen};
  \node[condition] (check) [below=of eingabe] {$n \geq 0$?};
  \node[output] (error) [right=2.5cm of check] {Fehler: Negative Zahl};
  \node[operation] (init) [below=of check] {$fak = 1$\\$i = 1$};
  \node[condition] (loop) [below=of init] {$i \leq n$?};
  \node[operation] (calc) [below=of loop] {$fak = fak \cdot i$\\$i = i + 1$};
  \node[output] (result) [below right=1.5cm and 2cm of loop] {Ergebnis: $fak$};
  \node[operation] (end) [below=of result] {Ende};

  % Pfeile zwischen den Knoten
  \draw[->] (start) -- (eingabe);
  \draw[->] (eingabe) -- (check);
  \draw[->] (check) -- node[left] {ja} (init);
  \draw[->] (check) -- node[above] {nein} (error);
  \draw[->] (init) -- (loop);
  \draw[->] (loop) -- node[left] {ja} (calc);
  \draw[->] (calc) -- ++(-2.5,0) |- (loop);
  \draw[->] (loop) -- node[above] {nein} (result);
  \draw[->] (error) |- (end);
  \draw[->] (result) -- (end);
\end{tikzpicture}
```

Ausgabe



6 Zusammenfassung

Die Erstellung von Programmablaufplänen mit Ti*k*Z bietet zahlreiche Vorteile, insbesondere die präzise Kontrolle über das Erscheinungsbild und die Konsistenz der Diagramme. Mit den in dieser Handreichung vorgestellten Grundtechniken können Sie bereits anspruchsvolle Flussdiagramme gestalten. Die Definition von Stilen für verschiedene Knotentypen mit `\tikzstyle` ermöglicht eine einheitliche Darstellung aller Elemente eines Typs. Die Platzierung von Knoten mit der `\node`-Funktion, sowohl mit absoluten Koordinaten als auch mit relativen Positionsangaben, bietet Flexibilität beim Aufbau des Diagramms. Durch die Erstellung von Verbindungspfeilen zwischen Knoten mit `\draw` und die Gestaltung komplexerer Pfade für Schleifen und Verzweigungen lassen sich auch anspruchsvolle Algorithmen anschaulich visualisieren. Diese Techniken sind zudem nicht auf Programmablaufpläne beschränkt, sondern können auch für andere Diagrammtypen wie Zustandsautomaten oder Netzwerkdiagramme verwendet werden. Für spezifischere Anforderungen bietet Ti*k*Z zahlreiche weitere Möglichkeiten, die Sie nach Bedarf erkunden können, doch die hier vorgestellten Grundlagen bilden ein solides Fundament für die gängigen Anwendungsfälle in der Informatik und Algorithmik.